

Explicit concept expressions

Thomas Köppe <tkoeppe@google.com>

Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Abstract

We propose to remove from the working paper the fact that mere application of arguments to concepts cause evaluation “everywhere” as boolean prvalue expressions and instead introduce a mildly more verbose “*simple-requires-expression*”. Instead of `bool b = Sortable<MyType>;` we propose `bool b = requires Sortable<MyType>;` This change removes unfortunate corner cases and paves the way for future extensions.

Contents

1. [Revision history](#)
2. [Before/After](#)
3. [Problem and motivation](#)
4. [Proposal](#)
5. [Alternatives](#)
6. [Context in relation to the Concepts TS Working Draft \(N4674\)](#)
7. [Impact on the past and future](#)
8. [Proposed wording](#)
9. [Acknowledgements](#)
10. [Discussion outcome: no consensus](#)

Revision history

- P1013R1 (this revision). A draft D1013R1 was [presented](#) to EWG in Rapperswil 2018, but the proposal did not achieve consensus. It is made available here for historic record. The draft is identical to this final revision except for this change entry and the discussion outcome below.

- More accurately describe the scope of the issue as applying to the use of concept names with template argument lists.
- Added description of the issue in the context of the Concepts TS and where it was going.
- Corrected description of the contexts where concept value evaluation remains just an *id-expression*.
- [P1013R0](#): Initial proposal

Before/After

Before the proposal	With the proposal
<pre>template <typename T> concept Foo = /* ... */; template <typename T> void f(T) { static_assert(Foo<T>); if constexpr (Foo<T>) { /* ... */ } }</pre>	<pre>template <typename T> concept Foo = /* ... */; template <typename T> void f(T) { static_assert(requires Foo<T>); if constexpr (requires Foo<T>) { /* ... */ } }</pre>

Problem and motivation

Consider a simple type concept:

```
template <typename T, typename U = T> concept Foo = /* ... */;
```

For a given type, say `int`, the current working paper makes `Foo<int>` a boolean prvalue. This creates a curious corner case if we consider future syntax extensions for abbreviated function templates. Consider the following code:

```
template <typename T> bool x(Foo<T>);
```

There are two plausible meanings this code could have (assuming further work in the direction of [P0745R0](#)):

```
template <typename T>                                bool x =
Foo<T>;    // #1 (variable)

template <typename T, typename U> requires Foo<T, U> bool x(U u);
    // #2 (function)
```

This is kind of an inverse of the “most vexing parse”. In the status quo of the working paper, the meaning is that of case #1 (a variable declaration); whereas at least some have expressed that the preferable meaning would be that of case #2 (a function declaration). This latter case is the one that is consistent with dropping constraints: `template <typename T> bool x(T)` is already a function declaration in the status quo.

We can avoid this problem if we make it so that applying template argument lists to plain concept names do not form prvalue expressions, thus spuriously occupying a “privileged” syntax. If we retain the status quo and later discover that this problem is serious, it will be a breaking change to remove or change the behaviour. On the other hand, if we remove the behaviour now and later discover that we actually do need it, we can easily add it back in without breaking code written to a level of C++ specified by an International Standard.

Proposal

We propose that an *id-expression* ([`expr.prim.id`]) that denotes the specialization of a concept no longer results in a prvalue except in specific contexts where we can expect normalization to apply the specific rule for normalization of an *id-expression* that names a specialization of a concept, i.e., it is not a subexpression of an atomic constraint (and in, for example, a *constraint-logical-or-expression*). To make up for the lost functionality, we introduce a new kind of expression: a “*simple-requires-expression*”, which turns a specialization of a concept into the boolean value indicating its constraint satisfaction.

Alternatives

Instead of making concepts not be expressions, we could make them expressions precisely when they are “named fully” or “fully specialized”, but treat them as

type placeholders when they are only “partially bound”. A hypothetical short function template syntax might then fit in like this:

```
template <typename A, typename B> concept Foo = /* ... */;

bool x(Foo<int, char>); // variable, equiv. to bool x = Foo<int,
char>;
bool y(Foo<char>);      // function, equiv. to template
<typename T> requires Foo<char, T> bool y(T)
```

In this approach, it is necessary to know the declaration of `Foo` in order to know whether `bool y(Foo<char>)` declares a variable or a function, with the usual concerns about maintenance, default arguments, etc.

This approach presupposes a detail of a future, not-yet-made design. It is a plausible approach, but at the same time this direction is compatible with our main proposal: we can first remove the evaluation-for-satisfaction interpretation of supplying arguments to concept names, and later bring it back with the semantics described here.

The proposed new *simple-requires-expression* is not strictly required, since a boolean value can also be obtained from an (ordinary) *requires-expression* such as “`requires { requires Foo<T>; }`”. We feel that such an expression would be somewhat unwieldy in contexts such as those of `constexpr if` and `static assertion`, and that the new *simple-requires-expression* will fit in more naturally.

Context in relation to the Concepts TS Working Draft (N4674)

The issue mentioned by this paper appears in the context of the Concepts TS as an ambiguity between *constrained-type-specifiers* whose *constrained-type-name* is a *partial-concept-id* and *simple-template-ids* whose *template-name* names a concept; the Concepts TS does not contain wording to add this ambiguity in either [stmt.ambig] or [dcl.ambig.res].

The Concepts TS has a process for concept resolution (N4674 [temp.constr.resolve]), the primary purpose of which is to handle overloading of function concepts, that could come into play as part of ambiguity resolution through determining whether the potential *partial-concept-id* is or is not a *type-name*. As an example, concept resolution will fail when considering `C<int>`

(below) as a placeholder type, because there is a mismatch between the `int` type argument and the non-type template parameter. The alternate interpretation of `C<int>` as an expression is considered viable.

```
template <typename T, int N = 0> concept bool C = true;
bool f(C<int>);
```

It is observed that semantic aspects in the details of the definition and of the use weigh into disambiguation with such a strategy.

We move beyond the traditional type-versus-expression ambiguity if we further pursue the direction, indicated to be intended by the design of the TS on the core reflector [\[core-28404\]](#), of allowing concept placeholders with other-than-type prototype parameters. On the reflector, it was observed that the TS admits *constrained-type-specifiers* (N4674 [dcl.type.simple]) to be placeholders for non-type and template template arguments, but does not extend the grammar to allow *constrained-type-specifiers* in the corresponding contexts; the latter situation appears to be considered a wording problem.

On that aspect of the design space, in addition to being short on wording to be found already in the Concepts TS, the GCC implementation behaves oddly when presented with such *constrained-type-specifiers* in contexts where they may have been parsed as a type [\[GccBug85846\]](#)[\[GccBug85991\]](#).

Additional design guidance would also be required on the value/value ambiguity between evaluation for substitution versus being a placeholder, and the interpretation as to whether an argument list forms part of a *partial-concept-id* or not when used with concepts having template prototype parameters.

For example, consider the following:

```
template <typename> class TmplA;

template <template <template <typename> class> class, template
<typename> class = TmplA>
concept bool Tmpl1 = true;

template <template <template <typename> class> class, typename =
int>
concept bool C = true;
```

```
bool f(C<Tmpl1<TmplA> >); // is Tmpl1<TmplA> a type-name or a
template-name?
```

We may find that these aspects hint towards needing extra syntax to engage the placeholder interpretation as opposed to requiring extra syntax for the evaluation-for-satisfaction; however, the future is hard to predict.

Impact on the past and future

There is no impact on the Standard, since the proposal modifies a feature that has not yet been standardized.

Removing concept *id-expressions* now means that we do not standardize a feature that we may later regret and cannot change without a break. On the other hand, adding the removed feature back in later, if and when we do need it, is straightforward.

Proposed wording

[to be fleshed out]

requires-expression:

requires *requirement-parameter-list*_{opt} *requirement-body*

simple-requires-expression:

requires *nested-name-specifier*_{opt} *simple-template-id*

An ~~*id-expression*~~ *simple-requires-expression* whose *simple-template-id* denotes the specialization of a concept [temp.concept] results in a prvalue of type **bool**. The expression is **true** if the concept's normalized *constraint-expression* [temp.constr.decl] is satisfied [temp.constr.constr] by the specified template arguments and **false** otherwise.

```
template<typename T> concept C = true;
static_assert(requires C<int>); // OK
```

[*Note*: A concept's constraints are also considered when using a template name [temp.names] and during overload resolution [over], and they are compared during the the partial ordering of constraints [temp.constr.order]. – *end note*]

Acknowledgements

Many thanks to Andrew Sutton for valuable discussion, Botond Ballo for necessary poking, and Richard Smith for his pre-Rapperswil review and feedback.

Discussion outcome: no consensus

This proposal was presented to EWG in Rapperswil 2018 but did not achieve consensus (SF: 4 | F: 15 | N: 11 | A: 6 | SA: 8). The main points of opposition were:

- The proposal adds undesired verbosity.
- The perceived problem is “hypothetical”, based on future proposals.
- There are mitigating factors for the potential of a breaking change from C++20:
 - The future terse syntax extensions may be free of the ambiguities we describe here.
 - Ambiguities, should they exist, may be addressed as part of processing the proposed terse syntax.
 - There is no breaking change to C++20 if terse syntax lands before C++20 ships.